#hackthekitty 2026

Project Report

Project Name: FeraMap

Reference ID: ZDO2HJYK

---

1. Executive Summary

FeraMap is a community-driven mobile app that turns stray cat sightings into coordinated TNR (trap-neuter-return) action. Community members spot and name cats on a live map, volunteers manage trapping and vet visits through a prioritized task queue, and colony coordinators track progress with real-time dashboards and exportable PDF reports, all built on real GPS data and AI-generated cat biographies that make every stray feel like a neighbour worth caring about.

2. Project Overview

2a. Why you're building what you're building

A few months before this hackathon, I was walking with a friend when a stray cat suddenly started following us. That isn't something stray cats usually do, so we stopped to check on her. She wasn't wearing a collar or any kind of identification, and there was no way to tell if she was lost or simply living on the streets.

I went to buy a small bag of cat food while my friend stayed with her. She ate everything almost immediately, which made it clear she hadn't eaten in a while. We tried contacting a local veterinarian, but we never got a response. We also reached out to an animal rescue organization, and they told us the best they could suggest was sharing her photo in a local WhatsApp group.

Eventually, we had to continue on our way. At some point she stopped following us, and we never saw her again. I still wonder what happened to her.

That experience stayed with me because it made me realize how difficult it is to actually help when you find a stray animal. There are people who care, volunteers who dedicate their time to helping cats, and organizations doing incredible work, but there isn't an easy way to connect someone who finds a cat with the people who can take action.

In Ciudad Lerdo, Durango, where I'm from, volunteers have been running trap-neuter-return (TNR) efforts for years using mostly WhatsApp groups, shared spreadsheets, and word of mouth. Those tools work, but they weren't designed for coordinating rescue efforts or keeping track of cats over time.

FeraMap was built to give that community a better way to organize information, share sightings, and work together to help stray cats more effectively.

2b. How it relates to the theme

The #hackthekitty theme calls for technology that makes the world better for cats and the communities that care for them. FeraMap does this directly and at multiple levels:

- Every stray cat gets a persistent digital identity: a name, a photo, a story written by the people who see them every day. They stop being anonymous.
- Volunteers stop coordinating by DM and start working from a shared, real-time task queue where cats are automatically prioritized by condition and urgency.
- Colony coordinators get the data they need to report to municipal authorities and donors.

- The community that already cares about these cats gets a tool that makes their care visible, coordinated, and lasting.

2c. Target Audience

FeraMap is designed for three interconnected user types:

1. **Community members**: neighbours, local residents, anyone who regularly sees stray cats in their area. They don't need to be part of a rescue organization. They just need a phone. They spot cats, name them, add photos, and log conditions. No training required.
2. **TNR volunteers**: the people who do the physical work of trapping, transporting to vets, and returning cats. FeraMap gives them a prioritized queue of cats that need attention, pre-loaded with community intelligence (where the cat hangs out, what it looks like, when it was last seen, whether it's injured).
3. **Colony coordinators and organizations**: the people managing TNR programs at scale. FeraMap gives them a live dashboard of neuter rates by neighbourhood zone and a one-tap PDF export they can share with municipal partners, vets, or donors.

The app is currently configured for Lerdo, Durango, México, but is designed to scale to any city. The GPS and PostGIS geofencing architecture is designed to support additional cities with coordinator-defined zone boundaries.

3. Key Features

☐ **Live community cat map**: real-time GPS map showing all known cats as colour-coded pins by TNR status. Tap any pin to open the cat's full profile.

☐ **5-step guided report flow**: GPS pin drop, cat identification, camera capture, condition logging, and photo upload to cloud storage.

☐ **AI-generated cat biographies**: community sighting notes are synthesized by Gemini 2.5 Flash into a readable personality description for each cat, making profiles shareable and emotionally engaging.

☐ **TNR trap queue with volunteer coordination**: prioritized list of cats needing attention, with self-assignment, status updates (spotted → trapped → neutered/returned), and condition tracking at each step.

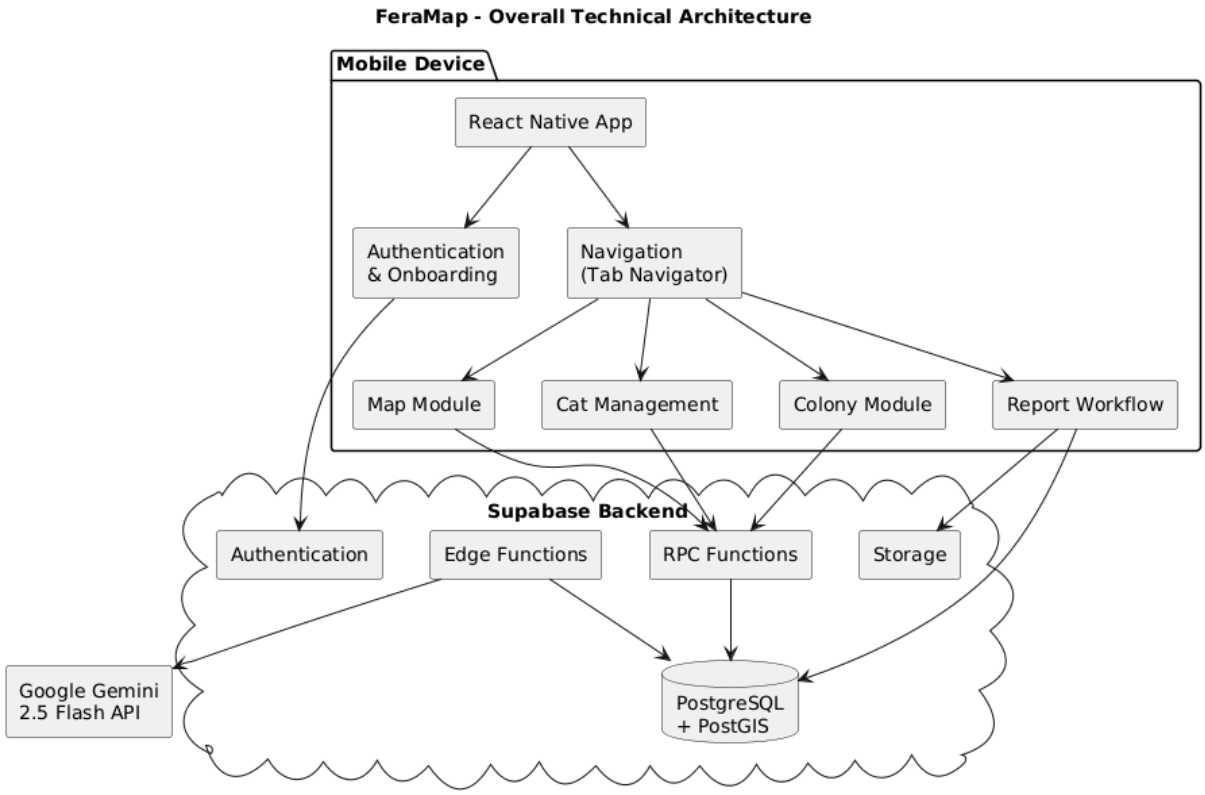
☐ **Animated polaroid photo stack**: multiple community photos stack on the cat profile in a swipeable polaroid style, giving each cat a visual story told by different people over time.

☐ **Colony progress dashboard + PDF export**: real-time neuter rates by neighbourhood zone, with one-tap PDF generation for reporting to municipal authorities or donors.

4. Technology Stack

Layer	Technology
Mobile framework	React Native + Expo SDK 54
Navigation	React Navigation 7
Backend	Supabase
Spatial data	PostGIS + Gist spatial index
Maps & location	react-native-maps + expo-location
AI integration	Google Gemini 2.5 Flash via Supabase Edge Function
Dev tooling	VS Code, Github, Figma, Affinity

5. Technical Architecture

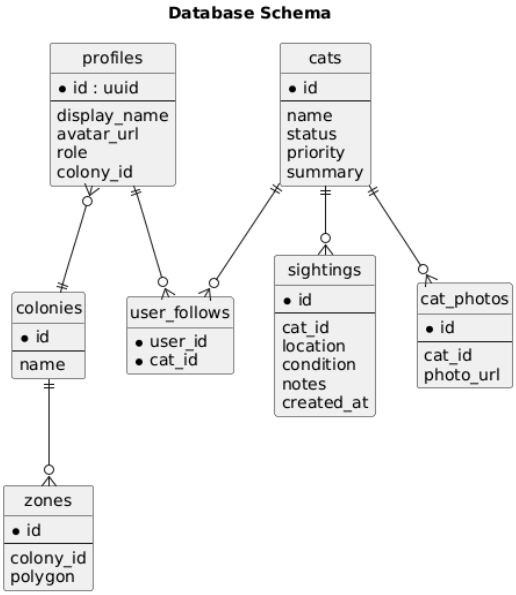


Navigation Structure

The app has three top-level states driven by auth and profile completion:

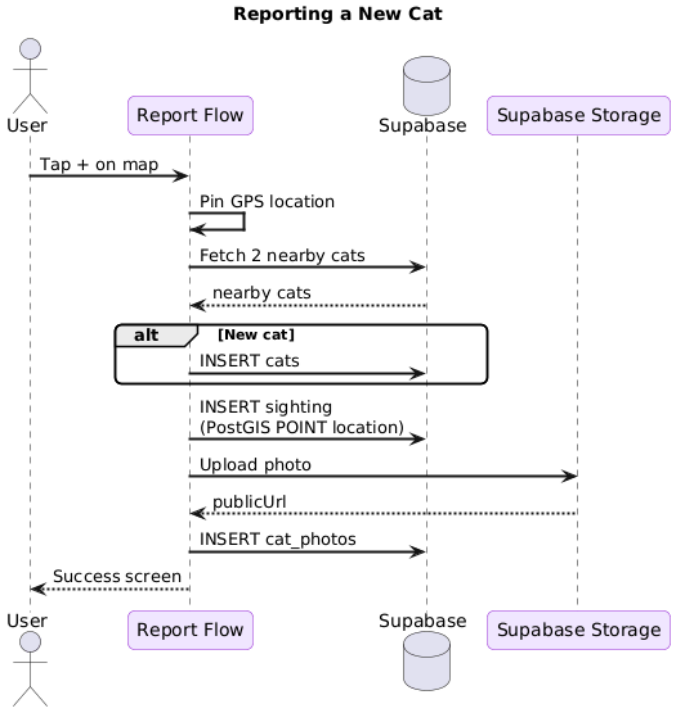
```
App.js
├── LoginScreen              (no session)
├── OnboardingScreen         (session but no profile)
└── RootStack                (session + profile)
    ├── TabNavigator
    │   ├── Map
    │   ├── Cats
    │   ├── Colony
    │   └── Profile
    ├── CatProfileScreen     (navigated from any tab)
    ├── TrapQueueScreen      (from Colony tab)
    └── Report (modal)
        └── Location → Identify → Camera → Details → Success
```

Database Schema

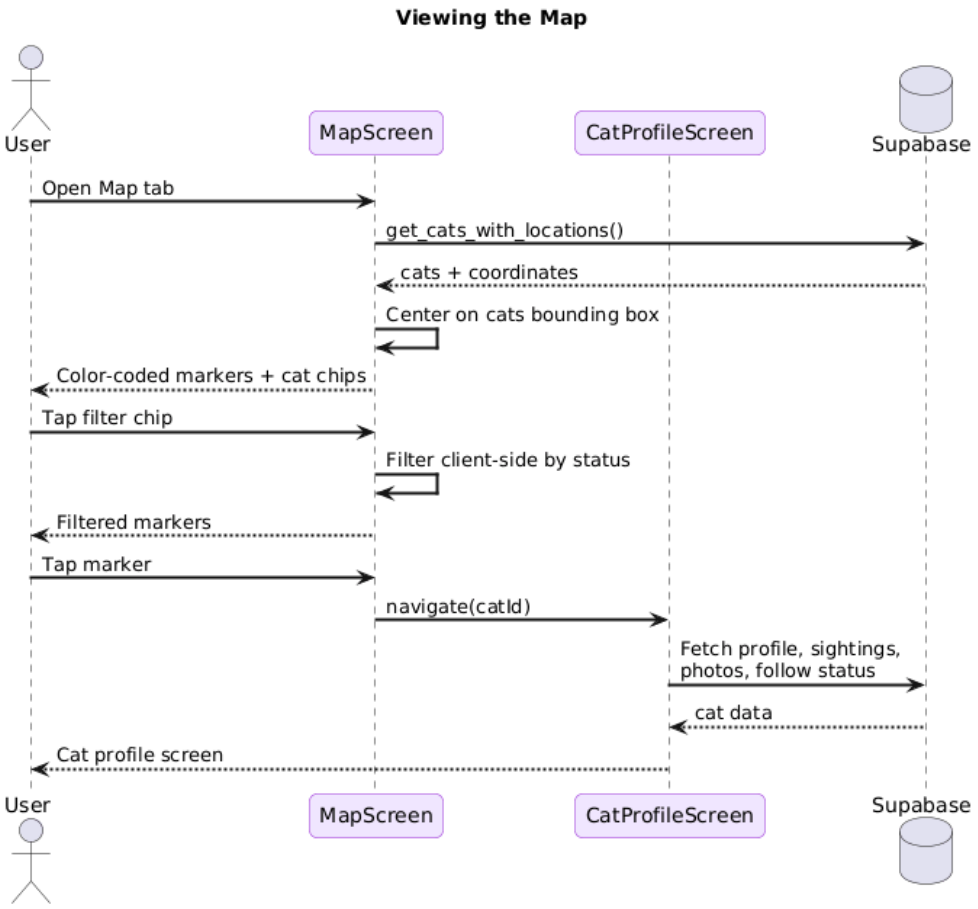


Key Data Flows

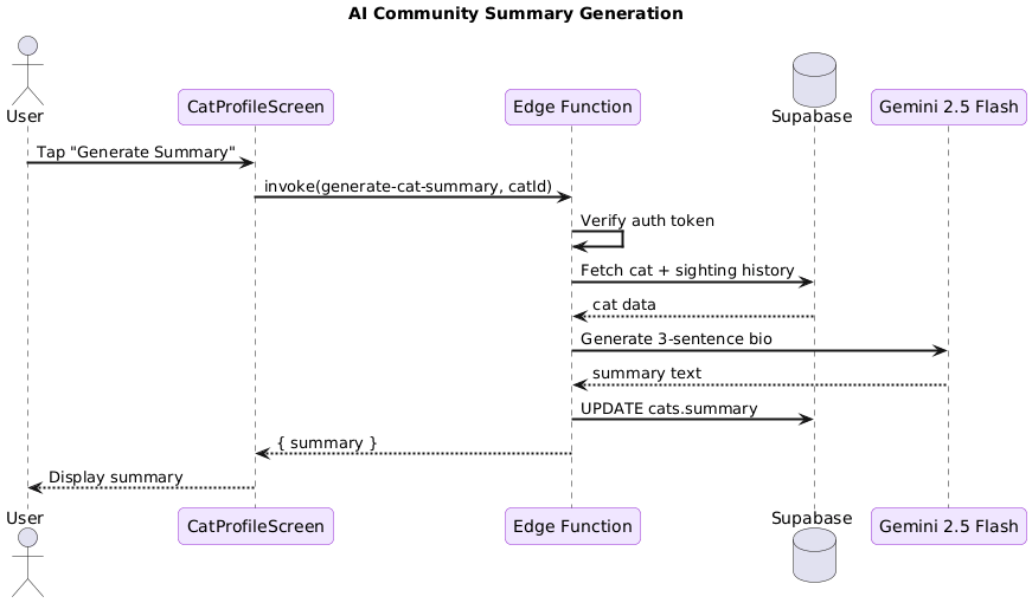
Reporting a new cat:
GPS pin (LocationScreen)
→ Identify existing or new cat (IdentifyScreen)
→ Photo capture (CameraScreen)
→ INSERT cats + INSERT sightings (PostGIS POINT) + upload photo to Storage
→ SuccessScreen



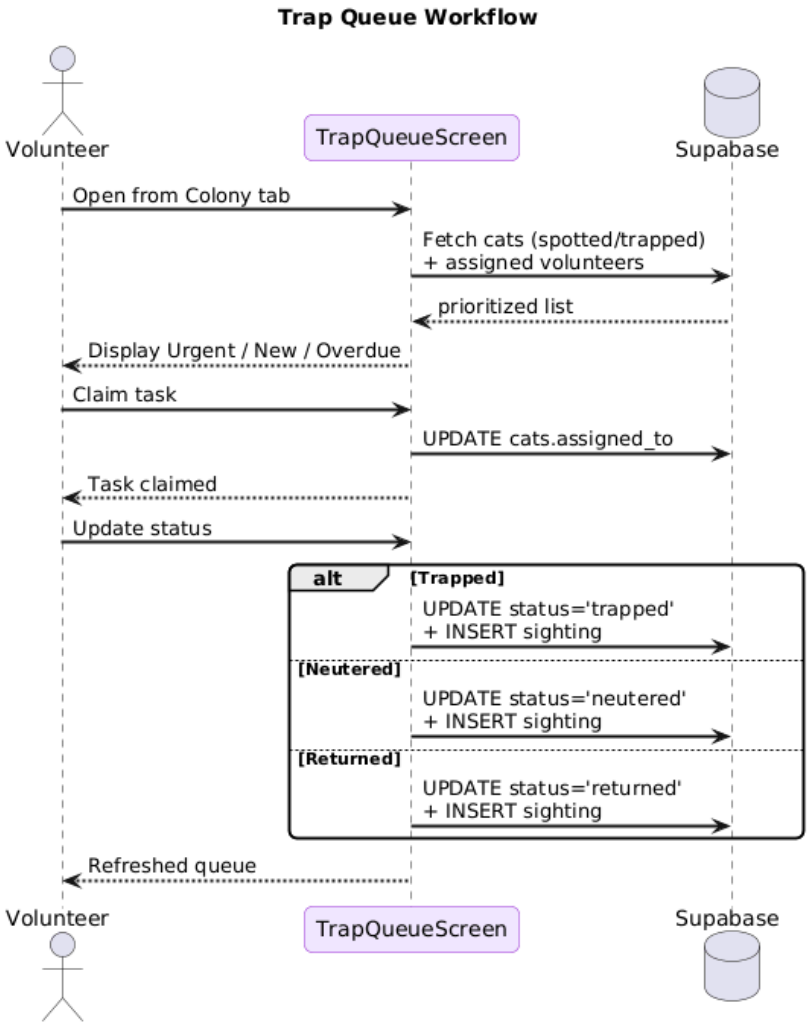
Viewing the map:
MapScreen → get_cats_with_locations() RPC
→ renders color-coded markers (spotted/trapped/neutered/returned)
→ tap marker → CatProfileScreen
→ get_cat_profile() + get_cat_sightings() RPCs
→ cat_photos table + user_follows check



AI community summary:
User taps "Generate summary" in CatProfileScreen
→ supabase.functions.invoke('generate-cat-summary', { catId })
→ Edge Function fetches sighting history
→ calls Google Gemini 2.5 Flash API
→ saves generated text to cats.summary
→ displayed in UI



Trap queue workflow:
ColonyScreen → TrapQueueScreen
→ fetch cats where status IN (spotted, trapped)
→ volunteer claims task → UPDATE cats.assigned_to
→ status update → INSERT sighting + UPDATE cat status



6. Testing Matrix

Feature / Flow	Steps	Expected Result	Actual Result	Pass / Fail
Sign up	Open app → tap "Make one" → enter email + password + confirm → tap "Create account"	Account created, onboarding screen appears	As expected	Pass
Login	Open app → enter email + password → tap "Login"	App opens to map screen	As expected	Pass
Forgot password	Login screen → enter email → tap "I forgot my password"	Alert confirms reset email sent	Reset email delivered to inbox. Reset link redirects to localhost, no in-app password reset screen implemented.	Partial
Onboarding — avatar	Complete signup → tap different cat avatars	Selected avatar shows purple ring, "You picked" label updates	As expected	Pass
Onboarding — colony	Onboarding screen → tap a community chip	Chip highlights in purple	As expected	Pass
Onboarding — submit	Fill name + select colony → tap "Let's Go!"	Profile saved, map screen appears	As expected	Pass
Map loads centered on cats	Open Map tab	Map centers on cat colony area, color-coded markers visible	As expected	Pass
Map filter	Map tab → tap "Neutered" filter chip	Only neutered cat markers remain visible	As expected	Pass
Map marker tap	Tap any cat marker on map	Callout appears with cat name	As expected	Pass
Report — location	Tap + FAB → drag map pin to location → tap Next	Location confirmed, moves to identify screen	As expected	Pass

Report — identify existing cat	Identify screen → select an existing cat → Next	Moves to camera with cat pre-selected	As expected	Pass
Report — new cat	Identify screen → select "New cat" → Next	Moves to camera, name field appears in details	As expected	Pass
Report — photo capture	Camera screen → tap camera button	Photo captured, moves to details screen	As expected	Pass
Report — photo from gallery	Camera screen → tap gallery icon → select photo	Photo selected, moves to details screen	As expected	Pass
Report — submit sighting	Details screen → enter name/condition/notes → tap "Report cat"	Sighting saved, success screen appears	As expected	Pass
Success screen — view profile	Success screen → tap "View full profile"	Cat profile screen opens	As expected	Pass
Success screen — back to map	Success screen → tap "Back to map"	Returns to map with new marker visible	As expected	Pass
Cat profile — photo stack	Open cat with multiple photos → tap photo stack	Photos cycle with animation	As expected	Pass
Cat profile — AI summary	Cat profile with no summary → tap summary box	Loading indicator appears, summary text generated and displayed	As expected	Pass
Cat profile — follow	Cat profile → tap "Follow"	Button changes to "Following"	As expected	Pass
Cat profile — unfollow	Cat profile → tap "Following"	Button reverts to "Follow"	As expected	Pass
Cat profile — see on map	Cat profile → tap "See on map"	Map opens centered on that cat with callout open	As expected	Pass
Cat profile — log sighting	Cat profile → tap "Log Sighting"	Report flow opens with cat pre-selected	As expected	Pass

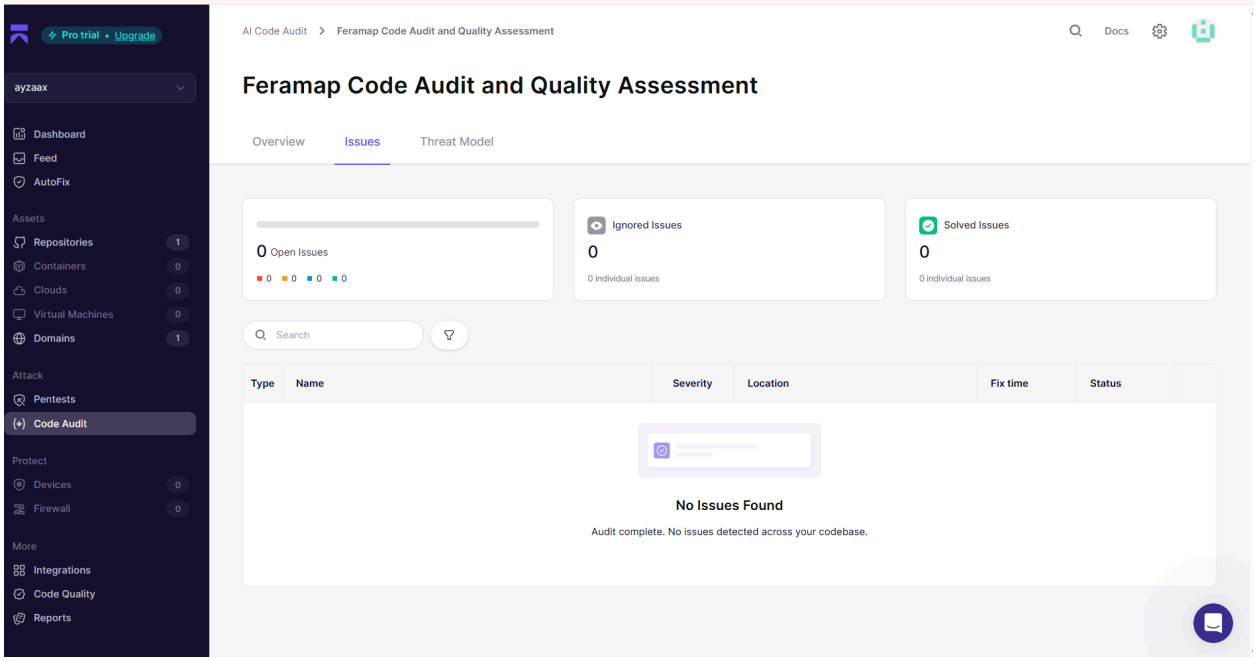
Cat profile — delete	Cat profile → tap "Delete this cat" → confirm	Cat removed, navigates back	As expected	Pass
Cats screen — search	Cats tab → type cat name in search bar	List filters to matching cats	As expected	Pass
Cats screen — filter	Cats tab → tap "Trapped" filter	Only trapped cats shown	As expected	Pass
Colony dashboard	Colony tab → view screen	Neuter rate stats and zone progress bars load	As expected	Pass
PDF export	Colony tab → tap export button	PDF generated and share sheet appears	As expected	Pass
Trap queue — view	Colony tab → tap "Trap Queue"	List of cats needing trapping loads with priorities	As expected	Pass
Trap queue — claim task	Trap queue → tap "Claim Task" on a cat	Cat assigned to current user	As expected	Pass
Trap queue — update status	Trap queue → tap "Update Status" → select "Neutered"	Cat status updates, sighting inserted	As expected	Pass
Profile — edit name	Profile tab → tap edit → change display name → save	New name appears on profile	As expected	Pass
Profile — change avatar	Profile tab → tap edit → select different avatar → save	New avatar displayed	As expected	Pass
Profile — change colony	Profile tab → tap edit → select different colony → save	Colony updated	As expected	Pass
Profile — followed cats	Profile tab → scroll to followed cats	Cats previously followed appear in list	As expected	Pass
Logout	Profile tab → tap "Log out"	Returns to login screen, session cleared	As expected	Pass

7. Security Review

FeraMap was designed with security as a core consideration throughout development. In addition to following secure development practices, the repository was evaluated using Aikido Security's AI Code Audit, the security scanning platform recommended by the HackTheKitty organizers.

The automated audit analyzed the project's source code for known vulnerabilities, insecure coding patterns, exposed secrets, dependency issues, and common security risks. The final scan completed successfully with no detected Critical, High, Medium, or Low severity vulnerabilities.

AI Security Audit Results



Security Measures Implemented

FeraMap incorporates multiple layers of security across the mobile application, backend services, and database.

Authentication

- User accounts are managed through **Supabase Authentication** using email and password credentials.
- Authentication sessions are handled by Supabase, allowing secure login, logout, and session persistence.
- Protected application features require an authenticated user.

Database Security

- Database access is protected using **Supabase Row Level Security (RLS)** to restrict access according to authenticated users and application policies.
- Database operations are performed through Supabase APIs and Remote Procedure Calls (RPCs), avoiding direct SQL execution from the client.
- Parameterized queries provided by Supabase help prevent SQL injection attacks.
- All seven application tables have Row Level Security enabled, with policies scoped to authenticated users and ownership rules enforced at the database level.

API and AI Security

- Integration with Google Gemini is performed through a **Supabase Edge Function** rather than directly from the mobile application.
- The Gemini API key remains securely stored on the server and is never exposed to client devices.

- Requests to the Edge Function require authenticated users, ensuring only authorized clients can invoke AI-powered features.
- Unauthenticated requests to the Edge Function receive a 401 response before any AI processing occurs.

Secrets Management

- Sensitive credentials are managed through environment variables.
- No private API keys, service-role credentials, or other secrets are committed to the project's source code repository.
- The client application only exposes the public Supabase URL and anonymous key, which are intended for client-side use and protected through backend security policies.

Cloud Storage

- Cat photographs are stored using Supabase Storage rather than locally within the application.
- Uploaded media is managed through authenticated backend services, providing centralized storage and access management.

Secure Development Practices

During development, several secure software engineering practices were followed, including:

- Version control using Git and GitHub.
- Separation of frontend and backend responsibilities.
- Server-side handling of privileged operations.
- Protection of API credentials through environment configuration.
- Automated security verification using Aikido Security prior to submission.

8. Future Improvements

☐ Push notifications: volunteers should receive alerts when a cat they follow changes status or a new cat is reported near their zone, without having to open the app.

☐ Offline mode: sightings captured without internet access should queue locally and sync when connectivity returns, since TNR fieldwork often happens in areas with poor signal.

☐ Municipal reporting dashboard: a web-based view for city officials or animal welfare organizations to see aggregated colony data without needing the mobile app.

☐ Multi-city support: the PostGIS and geofencing layer is already built for it; the next step is an admin panel to onboard new cities and their colony coordinators.

9. Tools You Used

☐ Figma: UI design and screen mockups.

☐ Affinity Designer: asset creation and logo design.

☐ Supabase Studio: database management, RLS policy testing, and SQL editor.

☐ Claude (Anthropic): AI coding assistant used throughout development for debugging, code review, and documentation.

- ☐ Google AI Studio: Gemini API key setup and prompt testing.
- ☐ Expo Go: live device testing on Android throughout development.
- ☐ GitHub: version control and pull request workflow.
- ☐ Aikido Security: automated security scanning of the repository.
- ☐ Canva: demo video editing.
- ☐ ElevenLabs: AI voice generator for demo video audio.

11. Learnings & Takeaways

Building FeraMap taught me that the hardest part of a civic tech project isn't the code but it’s understanding the real workflow of the people you're building for. TNR volunteers spend their time in the field, not behind a desk, so every design decision had to support the way they actually work.

On the technical side, integrating PostGIS into a mobile app taught me a lot about spatial data that I wouldn't have learned from a tutorial: how to index geography columns correctly, how RPC functions simplify complex spatial queries from the client, and why GPS coordinates need careful handling when you're computing bounding boxes dynamically.

On top of that, I learned that a clean git workflow pays off even in a solo hackathon project. Using feature branches, pull requests, and issue tracking made it much easier to isolate bugs and understand what changed when something broke.

12. Acknowledgments

I would like to thank the HackTheKitty organizers for creating a hackathon centered on a meaningful cause and encouraging participants to build technology that improves the lives of cats and the communities that care for them. I am also grateful to Supabase for providing the backend infrastructure that made it possible to build a production-ready application with authentication, database services, storage, and Edge Functions within the limited timeframe of a hackathon. Thanks to the Expo and React Native Maps teams for maintaining the open-source tools that power the mobile experience and mapping functionality, and to Google Gemini for enabling the AI-generated summaries that give each cat a unique story. I also appreciate Aikido Security for providing the automated security audit used to validate the application's security posture before submission.

Finally, this project is dedicated to the community cats who inspired FeraMap and to the volunteers who dedicate their time to improving their lives. Every stray cat deserves compassion, care, and the opportunity for a better quality of life, and I hope this project can contribute, even in a small way, to that mission.

Submission Checklist

- ☐ Video demo (HD or at least 720p)
- ☐ README.md (prerequisites, run instructions, configuration)
- ☐ Project report (this document, in documentation/)
- ☐ Source code in src/
- ☐ No unrelated files, executables, auto-generated code, or package folders (e.g. node_modules/, build/, dist/, vendor/)